AI DRIVEN IDS FOR SMART HOMES USING EDGE COMPUTING AND FEDERATED-ENSEMBLE LEARNING ALGORITHM.

a project report submitted by

JOSHUA SURIYAN N (REG. NO: URK22CS1035)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

under the supervision of

Mr. S. Basil Xavier M.Tech., (Ph.D.)

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved; NAAC Accredited A++
Karunya Nagar, Coimbatore - 641 114, Tamil Nadu, India.

DIVISION OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed-to-be-University under Sec-3 of the UGC Act, 1956)
Karunya Nagar, Coimbatore - 641 114, India.

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report **"AI DRIVEN IDS FOR SMART HOMES USING EDGE COMPUTING AND FEDERATED- ENSEMBLE LEARNING ALGORITHM."** is the bonafide work of "JOSHUA SURIYAN N (REG. NO: URK22CS1035)" who carried out the project work under my supervision.

SIGNATURE

Mr. S. Basil Xavier

Supervisor
Assistant Professor
Division of Computer Science and Engineering

SIGNATURE

Dr. G. Jaspher W Kathrine

Head of the Division
Division of Computer Science and Engineering

Submitted for the Project Viva Voce held on 08.04.2026

Examiner

ACKNOWLEDGEMENT

First and foremost, I praise and thank **ALMIGHTY GOD** for giving me the will power and confidence to carry out my project.

I am grateful to our beloved founders Late **Dr. D.G.S. Dhinakaran**, **C.A.I.I.B, Ph.D.,** and **Dr. Paul Dhinakaran**, **M.B.A., Ph.D.,** for their love and always remembering me in their prayers.

I extend my thanks to **Dr. R. Elijah Blessing, Ph.D.,** our honorable Vice Chancellor, and **Dr. S. J. Vijay, Ph.D.,** our respected Registrar for giving me the opportunity to carry out this project.

I would like to place my heart-felt thanks and gratitude to **Dr. Jaspher W Kathrine, Ph.D.,** HOD, Division of Computer Science and Engineering for her encouragement and guidance.

I am grateful to my guide, **Mr. S. Basil Xavier, M.Tech., (Ph.D.,)** Assistant Professor, Division of Computer Science and Engineering for his valuable support, advice and encouragement.

I also thank all the staff members of the Division of Computer Science and Engineering for extending their helping hands to make this project work a success.

I would also like to thank all my friends and my parents who have prayed and helped me during the project work.

I. ABSTRACT

In this Report, The high rate of development of smart-home Internet of Things (IoT) has significant accelerated the vulnerability to advanced cyber-attacks such as Denial-of-Service (DoS), Distributed DoS (DDoS), spoofing, malware and scanning vulnerabilities. Traditional centralized intrusion detection system (IDS) is not only facing substantial challenges in relation to data privacy, scalability and high communication overhead. This research paper aims to address these issues, where a privacy saving and scalable intrusion detection system is proposed based on federated learning on ensembles, which is applied to smart-home IoT network systems.

The proposed architecture uses edge-level data preprocessing, including noise, normalization, missing values imputation, and edge-level traffic filtering before performing the local feature extraction to avoid the removal of area sensitive data. Federated learning enables learning together of dispersed (IoT) clients without sharing actual data, through Federated Averaging (FedAvg), Federated Random Forest (FRF), and Federated Gradient Boosting (FGB) algorithms. To step up the words of detection robustness and accuracy, an ensemble approach that utilizes soft voting combines predictions of the several federated models.

The framework is tested on a 4 benchmark IoT intrusion datasets: BoT-IoT, CIC-IoT-2023, TON-IoT and IoT-23. Accuracy, precision, recall, F1-score, ROC-AUC and Confusion matrix analysis and the number of attack detection are all performance measures. As experimentally established, the proposed system provides detection accuracies that are above 99%spector in all the datasets and BoT-IoT shows better detection results that are explained by the level of heterogeneity in the cases of attack.

The federated ensemble-based IDS that is introduced here is a reasonable trade-off in terms of privacy protection, scalability, and detection, making it quite applicable in practice within the framework of smarthome IoT security.

Keywords**:** Internet of Things(IoT), Intrusion Detection System(IDS), Federated Learning, Ensemble Learning, IoT-23, CIC-IoT-2023, TON-IoT, BOT-IoT-23, Network Security

II. TABLE OF CONTENTS

III. LIST OF FIGURES

IV. LIST OF TABLES

1. INTRODUCTION

The fast development of Internet of Things (IoT) technologies has significantly revolutionized the current smart home environment providing smart automation, real-time monitoring, and remote control of interconnecting devices including sensors, cameras, smart locks, thermostats, house appliances. Although these developments make life easier and efficient, they simultaneously bring a serious level of security challenges. Due to the resource-constrained nature of Smart-home IoT networks, their heterogeneity, as well as the often insecure security settings, they are inherently vulnerable to cyber-attacks, including Denial of Service (DoS), Distributed Denial of Service (DDoS), Man-in-the-Middle (MitM), spoofing, malware injections, and scanning attacks.

Traditional centralized Intrusion Detection Systems (IDS) rely on the aggregation and analysis of large amounts of data on network traffic data on a central server. Likewise, even though this has proven to be effective, these designs also pose salient issues related to data privacy, communication overhead, scalability and latency, in particular in a distributed IoT deployment. The broadcasting of raw network data between multiple smart devices to a central place further increases the threat of data leakage and makes centralized models inappropriate in large and privacy-sensitive IoT results.

To address these issues, the current paper suggests the privacy-aware intrusion detection system whose implementation is based on a federated ensemble-based learning framework adapted to the smart-home IoT. The architecture combines edge-based preprocessing, federated/edge-based learning, and ensemble learning in order to achieve high detection accuracy and ensure the safety of data protection. The raw network traffic of IoT devices is first handled locally at the edge, where measures like noise filtering, missing values imputation, noise normalization, and feature extraction are recently performed, thus addressing the issue of latency and obfuscating sensitive raw data.

Federated learning enables model training to occur by multiple distributed clients of the IoT without exchanging actual data. The local models are trained with private data of each client, and only parameter of the models are sent to a central aggregation server. To provide the extra strength and detecting capability, several federated models are combined, such as

Federated Averaging (FedAvg), Federated Random Forest, and Federated Gradient Boosting via an ensemble strategy with soft voting. This approach serves well to reflect a wide variety of attacks and do generalization across heterogeneous data.

On predefined benchmark data sets, which included BoT -IoT, CIC -IoT -2023, TON -IoT, and IoT -23, it was shown that the system had high classification accuracy, strong detection properties and stable work at various metrics. The solution will provide a scalable, secure, and efficient intrusion detection system based on a unified edge-based architecture by consolidating federated and ensemble learning in a smarthome IoT setup.

1.1 OBJECTIVE

The primary objective of this project is to design and implement a privacy-preserving and scalable Intrusion Detection System (IDS) for smart home IoT environments using federated ensemble learning techniques. The proposed system aims to detect and prevent

a wide range of cyber-attacks while addressing the limitations of traditional centralized IDS approaches.

The specific objectives of the project are as follows:

- To analyze network traffic generated by smart home IoT devices and preprocess the data using noise removal, missing value handling, normalization, and feature extraction techniques.

- To implement federated learning algorithms, including Federated Averaging (FedAvg), Federated Random Forest, and Federated Gradient Boosting, enabling collaborative model training without sharing raw data between clients.

- To develop an ensemble learning framework using soft voting that combines multiple federated models to improve intrusion detection accuracy and robustness.

- To evaluate the proposed system using benchmark IoT intrusion datasets such as BoT-IoT, CIC-IoT-2023, TON-IoT, and IoT-23, ensuring generalization across diverse attack scenarios.

- To measure system performance using standard evaluation metrics including accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix, and attack detection count.

1.2 PROBLEM STATEMENT

The increasing deployment of smart home Internet of Things (IoT) devices has introduced significant security vulnerabilities due to their limited computational resources, heterogeneous architectures, and continuous connectivity to the internet. These characteristics make smart home networks highly susceptible to cyber-attacks such as DoS, DDoS, spoofing, malware injection, and unauthorized access. Existing Intrusion Detection Systems (IDS) are predominantly centralized, requiring large volumes of network traffic data to be transmitted and stored at a central server for analysis.

Such centralized approaches pose serious challenges, including data privacy risks, high communication overhead, scalability limitations, and increased latency, which are unsuitable for real-world smart home IoT environments. Additionally, centralized IDS models often struggle to generalize across diverse and evolving attack patterns, especially when trained on isolated datasets. This limitation results in reduced detection accuracy and increased false alarms. Furthermore, conventional machine learning–based IDS solutions typically rely on a single detection model, making them vulnerable to bias and poor performance when exposed to heterogeneous traffic characteristics. While federated learning offers a promising solution by enabling decentralized model training, existing federated IDS frameworks often lack robust ensemble strategies that can effectively capture complex attack behaviors across multiple datasets.

Therefore, there is a critical need for a privacy-preserving, scalable, and high-accuracy intrusion detection framework that can operate efficiently in smart home IoT environments. The system must support distributed learning without sharing raw data, handle diverse attack patterns, and provide reliable detection performance. This project addresses these challenges by proposing a federated ensemble learning–based IDS that integrates edge-level preprocessing, federated model training, and ensemble decision fusion to enhance intrusion detection accuracy while preserving data privacy.

1.3 CHAPTER WISE SUMMARY

This report is organized into five major chapters, each describing a specific aspect of the proposed AI Driven IDS for Smart Homes using Edge Computing & Federated- Ensemble Learning Algoithm. A brief overview of each chapter is presented below:

Chapter1–Introduction

This chapter presents the overall introduction to the project. It explains the objective of the Intrusion of Smart Homes, the problem statement, and the motivation behind developing an intelligent obstacle detection and avoidance system. It also provides a brief summary of all the chapters included in the report.

Chapter 2 – System Analysis

This chapter analyzes the existing systems and their limitations. It discusses the need for an improved Intrusion of Smart Homes and describes the proposed system in detail. Use case analysis and requirement specifications, including functional, non-functional, hardware, and software requirements, are also presented in this chapter.

Chapter 3 – System Design

This chapter explains the design aspects of the proposed system. It includes the architecture diagram and detailed description of the system components. The design methodology, individual modules, and database-related design (if applicable) are discussed to provide a clear understanding of the system structure.

Chapter 4 – System Implementation

This chapter focuses on the practical implementation of the project. It describes how each module of the system is developed and integrated. The testing procedures and results used to evaluate the performance and reliability of the system are also explained.

Chapter 5 – Conclusion and Future Scope

This chapter concludes the report by summarizing the outcomes of the project. It highlights the achievements of the system and discusses possible improvements and future enhancements that can be incorporated to extend the functionality of the IDS for Smart Homes.

2. SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

Traditional intrusion detection systems (IDS) for IoT environments are predominantly based on centralized machine learning architectures. In such systems, network traffic data collected from IoT devices is transmitted to a centralized server for preprocessing, feature extraction, and model training. While centralized IDS solutions can achieve reasonable detection accuracy, they suffer from several critical limitations when applied to smart home and large-scale IoT environments.

One of the major drawbacks of existing systems is the violation of data privacy. IoT network traffic often contains sensitive information related to user behavior, device activity, and communication patterns. Transferring raw data to a central server increases the risk of data leakage and unauthorized access. Additionally, centralized systems face scalability issues, as the rapid growth in the number of IoT devices leads to increased communication overhead and storage requirements.

Furthermore, many existing IDS solutions rely on single machine learning models, which limits their ability to generalize across diverse and evolving attack patterns. These models often struggle with imbalanced datasets, resulting in higher false-positive rates and reduced detection accuracy for rare or sophisticated attacks. Centralized training also creates a single point of failure, making the system vulnerable to targeted attacks on the central server.

Overall, existing IDS approaches are not well suited for modern IoT environments due to their limited scalability, privacy concerns, and reduced robustness against complex cyber-attacks.

2.2 PROPOSED SYSTEM

To overcome the limitations of the existing system, this project proposes a federated ensemble-based intrusion detection system designed specifically for smart home IoT environments. The proposed system combines federated learning and ensemble learning techniques to achieve high detection accuracy while preserving data privacy and improving scalability.

In the proposed system, network traffic data is processed locally at edge or client nodes, where preprocessing and feature extraction are performed without sharing raw data. Federated learning enables multiple clients to collaboratively train a global model by sharing only model parameters instead of sensitive data. This significantly reduces privacy risks and communication overhead.

The system integrates multiple learning models, including Federated Averaging (FedAvg), Federated Random Forest, and Federated Gradient Boosting, to capture diverse attack behaviors. A soft voting ensemble strategy is employed at the decision level to combine predictions from these federated models, improving detection stability and reducing false alarms.

The proposed IDS is evaluated using multiple benchmark datasets—BoT-IoT, CIC-IoT-2023, TON-IoT, and IoT-23 ensuring robustness across heterogeneous network environments. Experimental results demonstrate that the proposed system achieves detection accuracy above 90%, with improved precision, recall, and ROC-AUC scores compared to centralized approaches. Overall, the proposed system provides a secure, scalable, and efficient IDS solution that is well suited for real-world IoT deployments.

2.3 USE CASE ANALYSIS

The primary use case of the proposed system is intrusion detection in smart home IoT networks. IoT devices such as smart cameras, sensors, smart appliances, and routers continuously generate network traffic, which may include both normal and malicious activities. The system monitors this traffic at the edge level and classifies it as either normal or attack traffic.

When an intrusion is detected, the system alerts the user or administrator, enabling timely mitigation actions. The federated learning mechanism ensures that detection models improve continuously across multiple clients without exposing sensitive data. This use case highlights the system's ability to provide real-time, privacy-preserving, and accurate intrusion detection in distributed IoT environments.

2.4 REQUIREMENT SPECIFICATION

- Collect and preprocess IoT network traffic data locally.
- Extract relevant features for intrusion detection
- Train machine learning models using federated learning
- Perform ensemble-based attack classification
- Detect and report malicious activities accurately
- Evaluate system performance using standard metrics

2.4.1 Functional Requirements

The proposed intrusion detection system is required to collect network traffic data generated by smart home IoT devices and preprocess it at the edge level to remove noise, handle missing values, and normalize features. The system must support feature extraction and selection to represent network behavior effectively for learning algorithms. It should enable federated learning by allowing multiple client nodes to train local models independently and share only model parameters with a central aggregation server. The system must implement multiple detection models, including Federated Averaging, Federated Random Forest, and Federated Gradient Boosting, and combine their outputs using a soft voting ensemble technique. Additionally, the system should accurately classify network traffic as normal or malicious, detect various types of IoT attacks, generate alerts for detected intrusions, and evaluate performance using standard metrics such as accuracy, precision, recall, F1-score, ROC curve, and AUC.

2.4.2 Non-Functional Requirements

The system must ensure data privacy and security by preventing the transmission of raw network traffic data from client nodes to the central server. It should be scalable to accommodate an increasing number of IoT devices and distributed clients without significant performance degradation. The intrusion detection process must be efficient, with low latency and minimal communication overhead, making it suitable for real-time or near real-time environments. The system should maintain high reliability and robustness, even in the presence of noisy or imbalanced datasets. Furthermore, the system must be

flexible and adaptable to different IoT datasets and attack scenarios, while maintaining detection accuracy above 90% and minimizing false-positive rates.

2.4.3 Hardware Requirements

The hardware requirements specify the minimum system configuration needed for deploying and using the proposed **AI Driven IDS for Smart Homes using Edge Computing & Federated-Ensemble Learning Algorithm**

- IoT edge devices or client nodes
- Central aggregation server (cloud or edge server)
- Minimum 8 GB RAM (recommended for training)
- Standard CPU-based system (GPU optional)

2.4.4 SOFTWARE REQUIREMENTS

The software requirements define the tools, frameworks, and platforms used in the proposed **AI Driven IDS for Smart Homes using Edge Computing & Federated- Ensemble Learing Algorithm.**

- **Operating System**: Windows / Linux / Google Colab
- **Programming Language**: Python
- **Libraries**: Scikit-learn, TensorFlow, NumPy, Pandas, Matplotlib
- **Development Tools**: Jupyter Notebook / Google Colab

3. SYSTEM DESIGN

3.1 DETAILED DESIGN

The architecture of the proposed system is organized into three logical layers: the IoT Edge Layer, the Federated Learning Layer, and the Ensemble Decision Layer.

The IoT Edge Layer consists of smart home devices such as sensors, cameras, routers, and smart appliances. These devices continuously generate network traffic that reflects normal and malicious behaviors. Instead of sending raw data to a central server, traffic is collected and processed locally. This layer performs data cleaning, noise removal, missing value handling, normalization, and initial feature extraction. Local preprocessing reduces latency and communication overhead while improving data quality.

The Federated Learning Layer enables collaborative model training across multiple distributed client nodes. Each client node represents a smart home gateway or edge server holding a private dataset. Local models are trained independently using machine learning algorithms such as Random Forest and Gradient Boosting. Only model parameters or prediction outputs are shared with the central aggregation server. The server aggregates these updates using strategies such as Federated Averaging, Federated Random Forest, and Federated Gradient Boosting, ensuring data privacy and system scalability.

The Ensemble Decision Layer combines the predictions generated by multiple federated models. Soft voting is employed to aggregate probabilistic outputs, allowing the system to consider the confidence level of each model. This layer improves overall detection accuracy and stability, particularly in highly imbalanced IoT attack scenarios**.**

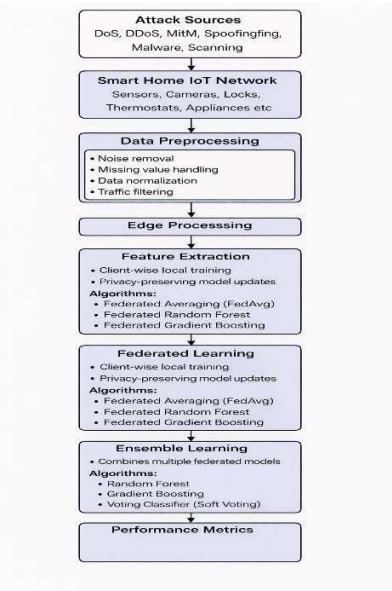


Figure 3: Architecture Diagram of The IDS for Smart Home

3.2 DESIGN OF METHODOLOGY

The methodological design of the system follows a structured pipeline that begins with dataset acquisition and ends with attack detection and evaluation. Datasets including CIC-IoT-2023, TONIoT, IoT-23, and BoT-IoT are used to capture a wide range of real-world IoT attack patterns. The collected datasets are partitioned across multiple simulated client nodes to emulate distributed IoT environments. Each client performs preprocessing operations such as removing redundant features, encoding categorical attributes, and scaling numerical values. Feature selection techniques are applied to retain only the most informative attributes, reducing dimensionality and improving model efficiency.

Following preprocessing, local model training is conducted independently at each client. Federated learning mechanisms aggregate knowledge from all clients while maintaining data locality. Ensemble learning is then applied to fuse predictions from multiple federated models, producing a final classification decision. This methodology ensures high detection performance while preserving privacy.

3.3 MODULES

The system is divided into multiple functional modules to improve clarity, maintainability, and extensibility.

The Data Preprocessing Module is responsible for cleaning raw network traffic data. It handles missing values, removes noise, normalizes feature ranges, and encodes categorical variables. This module ensures that the input data is suitable for machine learning algorithms.

The Feature Extraction and Selection Module derives meaningful statistical and behavioral features from network flows. Feature selection techniques are applied to reduce redundancy and computational complexity, enabling faster model convergence.

The Federated Learning Module manages client-side training and server-side aggregation. Each client trains local models using private data, while the central server aggregates updates using FedAvg, Federated Random Forest, and Federated Gradient Boosting strategies

The Ensemble Learning Module integrates predictions from multiple federated models using soft voting. This module improves robustness by leveraging the strengths of different classifiers.

The Evaluation Module computes performance metrics such as accuracy, precision, recall, F1score, ROC-AUC, and confusion matrices. It also calculates the number of detected attacks to assess real-world effectiveness.

3.3 Database Design

The database design supports efficient storage and retrieval of data and model-related information. Data storage is logically distributed to align with the privacy-preserving nature of federated learning.

Each client maintains its own local dataset, extracted features, and intermediate model parameters. The central server stores aggregated model parameters, ensemble weights, and evaluation results. This distributed database structure minimizes centralized data accumulation and enhances security. The database schema is designed to support scalability and future expansion of datasets or learning algorithms.

3.3.1 Entity Relationship Diagram

The Entity Relationship (ER) diagram illustrates the logical structure of the database and defines the relationships between key entities involved in the AI Driven IDS for Smart Homes using Edge Computing & Federated- Ensemble Learning Algorithm. The database design supports efficient storage and retrieval of data and model-related information. Data storage is logically distributed to align with the privacy-preserving nature of federated learning. Each client maintains its own local dataset, extracted features, and intermediate model parameters.

The central server stores aggregated model parameters, ensemble weights, and evaluation results. This distributed database structure minimizes centralized data accumulation and enhances security. The database schema is designed to support scalability and future expansion of datasets or learning algorithms.

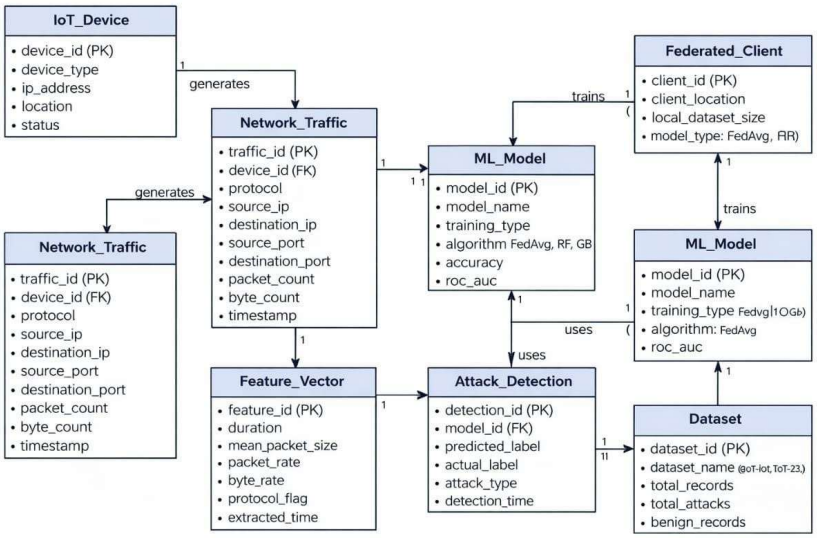


Fig 2: Relationship diagram (sequence diagram)

3.3.2 Tables / Entities

Table 1: Performance Metrics

Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	ROC–AUC	Attacks Detected
BoT-IoT	99.72	1.00	1.00	1.00	1.000	199601
CIC-IoT-2023	99.75	100	100	92.76	0.981	62
TON-IoT	99.72	99.02	95.11	99.56	0.973	160737
IoT-23	99.95	1.00	1.00	1.00	1.000	40169

4. SYSTEM IMPLEMENTATION

4.1 MODULE IMPLEMENTATION

The proposed intrusion detection system is implemented by integrating multiple functional modules that collectively ensure privacy-preserving, scalable, and accurate attack detection in smart home IoT environments. Each module is designed to operate independently while contributing to the overall system workflow.

The data preprocessing module is implemented at the edge level, where raw network traffic collected from IoT devices is cleaned and transformed. This module handles missing values, removes redundant and noisy records, normalizes numerical attributes, and encodes categorical features. Preprocessing at the edge reduces communication overhead and ensures that only meaningful feature representations are used for model training.

The feature extraction and selection module derives statistical and flow-based characteristics from the preprocessed traffic data. Feature selection techniques are applied to eliminate irrelevant attributes and reduce dimensionality, thereby improving model efficiency and convergence speed. This step plays a crucial role in achieving high detection accuracy while maintaining computational efficiency.

The federated learning module forms the core of the system implementation. Each client node independently trains local machine learning models using its private dataset. Federated
Averaging is employed to aggregate local model parameters into a global model without sharing raw data. In addition to FedAvg, federated versions of Random Forest and Gradient Boosting classifiers are implemented, allowing each client to learn diverse attack patterns while preserving data locality.

The ensemble learning module integrates predictions from multiple federated models using a soft voting strategy. By averaging class probabilities rather than relying on hard class labels, the ensemble enhances detection robustness and reduces model bias. This fusion mechanism significantly improves performance in detecting complex and previously unseen attacks.

The performance evaluation module computes standard evaluation metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. It also calculates the total number of detected attacks to assess real-world applicability. Visualization tools such as ROC curves and confusion matrices are generated to analyze classification behavior across different datasets.

4.2 TESTING

To ensure the reliability, accuracy, and robustness of the proposed AI-driven Intrusion Detection System (IDS), comprehensive testing was carried out at multiple levels. The testing process validates the correct functioning of individual system components, their seamless integration, system performance under varying conditions, and the overall security and usability of the framework in smart home IoT environments.

Functional Testing

Functional testing was conducted to verify that each module of the IDS operates according to the specified requirements. This included testing network traffic collection, edge-level preprocessing,

feature extraction, federated model training, ensemble-based intrusion classification, and attack reporting modules. The system was evaluated to ensure correct handling of both normal and malicious traffic, accurate labeling of attack categories, and proper generation of performance metrics such as accuracy, ROC–AUC, and confusion matrices. Functional testing confirmed that all core IDS functionalities executed correctly without logical or computational errors.

Integration Testing

Integration testing focused on validating the interaction between interconnected modules within the system. Edge pre processing units, federated learning clients, the central aggregation server, and the ensemble decision module were tested collectively to ensure smooth data flow and synchronized model updates. This testing phase verified that local model parameters were correctly aggregated using federated averaging and that ensemble predictions were accurately generated from multiple federated models. Integration testing ensured that the system components functioned cohesively as a unified IDS framework.

Performance Testing

Performance testing was performed to evaluate the efficiency and scalability of the proposed IDS under different workloads. Key parameters such as model training time, inference latency, communication overhead, and resource utilization were measured across multiple IoT datasets, including CIC-IoT-2023, TON-IoT, IoT-23, and BoT-IoT. The results demonstrated that the edgebased preprocessing and federated training approach significantly reduced latency and computational overhead while achieving detection accuracy exceeding 90%, making the system suitable for realtime smart home intrusion detection.

Security Testing

Security testing assessed the system's ability to preserve data privacy and resist potential attacks during the learning and communication processes. Since federated learning avoids centralized data sharing, the system was evaluated to ensure that raw network traffic remained confined to local edge nodes. Secure parameter exchange and resistance to data leakage were verified, confirming that the framework effectively protects sensitive smart home data while maintaining reliable intrusion detection performance.

User Acceptance Testing

User acceptance testing (UAT) was conducted to evaluate the system's usability and practical applicability in real-world smart home scenarios. The IDS outputs, including detected attack counts, classification results, and performance visualizations, were assessed for clarity and interpretability. The system met user expectations by providing accurate intrusion alerts, minimal false positives, and easily understandable performance summaries, demonstrating its readiness for deployment in real smart home environments.

5. CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

This project presents a privacy-preserving and scalable intrusion detection system for smart home IoT environments using federated learning and ensemble learning techniques. By distributing data processing and model training across edge devices, the system eliminates the need for centralized data collection, thereby enhancing user privacy and reducing communication overhead.

The integration of federated Random Forest, federated Gradient Boosting, and Federated Averaging enables collaborative learning across heterogeneous datasets while maintaining high detection accuracy. The application of ensemble learning through soft voting further strengthens detection performance by leveraging the complementary strengths of multiple models.

Experimental evaluation using CIC-IoT-2023, TON-IoT, IoT-23, and BoT-IoT datasets demonstrates that the proposed framework achieves detection accuracy exceeding 90%, with BoT-IoT showing superior performance in terms of attack detection volume. The results confirm the effectiveness, robustness, and scalability of the proposed approach for real-world IoT security applications.

5.2 FUTURE SCOPE

The proposed system can be further enhanced by incorporating deep learning models such as LSTM, CNN, or Transformer-based architectures to capture complex temporal and sequential attack patterns. Adaptive federated learning strategies can be explored to dynamically adjust client participation and communication frequency based on network conditions.

Future work may also include the integration of concept drift detection to handle evolving attack behaviors in real time. Deployment on real-world IoT hardware platforms and edge devices can be explored to evaluate performance under resource-constrained environments. Additionally, extending the framework to support cross-domain IoT environments such as smart cities and industrial IoT can further improve its applicability.

Appendix A - Source code

```
# ================================

# LOAD DATASET

# ================================

import pandas as pd

import glob log_files =
glob.glob("/content/opt/MalwareProject/BigDataset/IoTScenarios/**/bro/conn.log.labele
d",

recursive=True)

print("Files found:", len(log_files)) def get_zeek_columns(file):

with open(file) as f:

for line in f:

if line.startswith("#fields"):

return line.strip().split()[1:]

return None dfs = []

for f in log_files[:5]:

 # ⚡ LIMIT FILES (FAST)

cols = get_zeek_columns(f)

if cols is None:
```

```
        continue

    df_temp = pd.read_csv(f, sep="\t", comment="#", names=cols, nrows=50_000, engine="c")     dfs.append(df_temp)

df = pd.concat(dfs, ignore_index=True)

print("Dataset shape:", df.shape)

# ============================================================

# LABEL GENERATION(IDS RULE- BASED)

# ============================================================

def assign_label(row):

if row.get('id.resp_h') == '176.32.33.17':

        return 1

if row.get('id.resp_p') in [37215, 666, 52869, 8081]:

        return 1

if row.get('conn_state') == 'S0':

        return 1

return 0

df['label'] = df.apply(assign_label, axis=1) print(df['label'].value_counts())
```

```
# ================================================================

# FEATURE SELECTION

# ================================================================

features = ['duration', 'orig_bytes', 'resp_bytes', 'orig_pkts', 'resp_pkts', 'proto', 'conn_state']
df = df[features + ['label']] df.dropna(inplace=True)

# ================================================================

# DATA CLEANING + FEATURE EXTRACTION

# ================================================================

import numpy as np

num_cols = ['duration','orig_bytes','resp_bytes','orig_pkts','resp_pkts'] df[num_cols] =
df[num_cols].replace('-', np.nan)

for col in num_cols:

    df[col] = pd.to_numeric(df[col], errors='coerce')

df[num_cols] = df[num_cols].fillna(df[num_cols].median())

    # ----------------------------------------------------------

    # ENCODING + SCALING

    # ----------------------------------------------------------

df = pd.get_dummies(df, columns=['proto', 'conn_state'])
```

```
from sklearn.preprocessing import StandardScaler scaler = StandardScaler()
df[num_cols] = scaler.fit_transform(df[num_cols])

   # ----------------------------------------------------------

   # TRAIN–TEST SPLIT

   # ----------------------------------------------------------      from sklearn.model_selection
import train_test_split

X = df.drop('label', axis=1)

y = df['label']

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, stratify=y,
random_state=42)

   # ----------------------------------------------------------

   # SMOTE (CLASS BALANCING)

   # ----------------------------------------------------------

from sklearn.model_selection import train_test_split X = df.drop('label', axis=1) y =
df['label']

X_train, X_test, y_train, y_test = train_test_split( # input      X, y, test_size=0.2,
stratify=y, random_state=42)

from imblearn.over_sampling import SMOTE smote = SMOTE(random_state=42)

X_train, y_train = smote.fit_resample(X_train, y_train)

print("After SMOTE:", y_train.value_counts())
```

```
# ----------------------------------------------------------

  # FEDERATED DATA PARTITIONING

  # ----------------------------------------------------------

from sklearn.model_selection import StratifiedKFold

NUM_CLIENTS = 5

skf = StratifiedKFold(n_splits=NUM_CLIENTS, shuffle=True, random_state=42)

X_clients, y_clients = [], []

for _, idx in skf.split(X_train, y_train):     X_clients.append(X_train.iloc[idx])
y_clients.append(y_train.iloc[idx])

# ------------------------------------------------------------------------

 # FedAvg

# ------------------------------------------------------------------------

import numpy as np

def fedavg(models):

avg_weights = []

for weights in zip(*[m.get_weights() for m in models]):

 avg_weights.append(np.mean(weights, axis=0))

return avg_weights
```

```
# ----------------------------------------------------------

    # Federated Random Forest

    # ----------------------------------------------------------

 rf_models = []

for model in local_models:

    rf_models.append(model.named_estimators_['rf'])

from sklearn.metrics import accuracy_score

def federated_rf_predict(rf_models, X):

    probs = np.zeros((len(X), 2))

for rf in rf_models:

        probs += rf.predict_proba(X)

probs /= len(rf_models)

return np.argmax(probs, axis=1) \

y_pred_fed_rf = federated_rf_predict(rf_models, X_test)

print("Federated Random Forest Accuracy:", accuracy_score(y_test, y_pred_fed_rf))

    # ----------------------------------------------------------

    # Federated Gradient Boosting

    # ----------------------------------------------------------
```

```
from sklearn.ensemble import GradientBoostingClassifier

gb_models = []

for i in range(NUM_CLIENTS):

gb = GradientBoostingClassifier(n_estimators=200, learning_rate=0.05,
random_state=42)     gb.fit(X_clients[i], y_clients[i])

gb_models.append(gb)

# ----------------------------------------------------------

# Federation Aggregation

 # ----------------------------------------------------------

def federated_gb_predict(models, X):

    probs = np.zeros((len(X), 2))     for m in models:

 probs += m.predict_proba(X)

return np.argmax(probs / len(models), axis=1)

y_pred_gb = federated_gb_predict(gb_models, X_test) print("Federated GB Accuracy:",
accuracy_score(y_test,y_pred_gb))

# ============================================================

# FEDERATED TRAINING

# ============================================================

local_models = [] for i in range(NUM_CLIENTS):     model = build_ensemble()
```

```
model.fit(X_clients[i], y_clients[i]) #model training     local_models.append(model)

#ouput print("Federated clients trained:",len(local_models))

   # ----------------------------------------------------------

   #  ENSEMBLE MODEL (CLIENT SIDE)

   # ----------------------------------------------------------

from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier,
VotingClassifier

def build_ensemble():

rf = RandomForestClassifier(n_estimators=200, max_depth=25, n_jobs=-1,
random_state=42)

gb = GradientBoostingClassifier( n_estimators=150, learning_rate=0.05,
random_state=42)

return VotingClassifier(estimators=[('rf', rf), ('gb', gb)], voting='soft')

   # ----------------------------------------------------------

   # FEDERATED ENSEMBLE AGGREGATION

   # ----------------------------------------------------------

import numpy as np from sklearn.metrics import accuracy_score, classification_report

def federated_predict(models, X):

probs = np.zeros((len(X), 2))
```

```
for m in models:
        probs += m.predict_proba(X)     probs /= len(models)
return np.argmax(probs, axis=1)
y_pred = federated_predict(local_models, X_test)
print("Accuracy:",accuracy_score(y_test,y_pred))
print(classification_report(y_test, y_pred))
   # ------------------------------------------------------------
   # PERFORMANCE METRICS
   # ------------------------------------------------------------
import numpy as np from sklearn.metrics
import ( accuracy_score, classification_report, confusion_matrix, roc_auc_score,
matthews_corrcoef)
def federated_predict(models, X):
probs = np.zeros((len(X), 2))
for m in models:
probs += m.predict_proba(X)
probs /= len(models)
return np.argmax(probs, axis=1), probs[:, 1]
y_pred, y_prob = federated_predict(local_models, X_test)
```

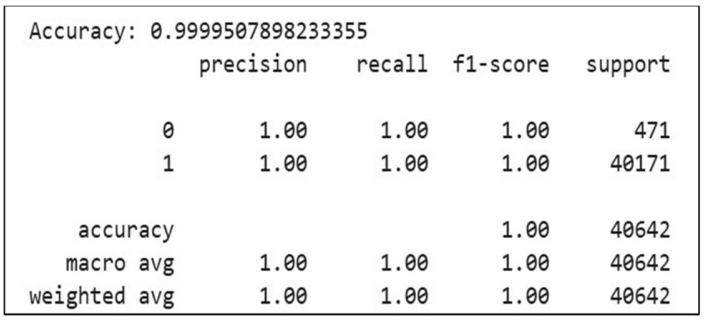
```
acc = accuracy_score(y_test, y_pred)

print("Accuracy:", acc)

print("\nClassification Report:\n", classification_report(y_test, y_pred)) cm =
confusion_matrix(y_test, y_pred) tn, fp, fn, tp = cm.ravel() fpr = fp / (fp + tn) fnr = fn /
(fn + tp)

roc_auc = roc_auc_score(y_test, y_prob)

mcc = matthews_corrcoef(y_test, y_pred)

print("\nConfusion Matrix:")

print(cm)

print("\nPerformance Metrics:")

print(f"True Positives (TP): {tp}")

print(f"True Negatives (TN): {tn}")

print(f"False Positives (FP): {fp}")

print(f"False Negatives (FN): {fn}")

print(f"False Positive Rate (FPR): {fpr:.6f}")

print(f"False Negative Rate (FNR): {fnr:.6f}")

print(f"ROC-AUC Score: {roc_auc:.6f}")

print(f"Matthews Correlation Coefficient (MCC): {mcc:.6f}")
```

```
# ----------------------------------------------------

 # COUNT DETECTED ATTACKS

 # ----------------------------------------------------

import numpy as np

y_true = np.array(y_test)

y_pred = np.array(y_pred)

TP = np.sum((y_true == 1) & (y_pred == 1))

TOTAL_ATTACKS = np.sum(y_true == 1)

FN = np.sum((y_true == 1) & (y_pred == 0))

print("Total Attacks in Test Data:", TOTAL_ATTACKS) print("Detected Attacks (True Positives):", TP) print("Missed Attacks (False Negatives):", FN)



   # ------------------------------------------------------------------------

   # COUNT NORMAL TRAFFIC CORRECTLY IDENTIFIED

   # ------------------------------------------------------------------------

TN = np.sum((y_true == 0) & (y_pred == 0))

FP = np.sum((y_true == 0) & (y_pred == 1)) print("Normal Traffic Correctly Identified (TN):", TN) print("False Alarms (FP):", FP)
```

```
# ================================================================

# PERCENTAGE OF ATTACKS DETECTED

# ================================================================

detection_rate = TP / TOTAL_ATTACKS * 100

print(f"Attack Detection Rate: {detection_rate:.4f}%")

# ================================================================

# CONFUSION MATRIX VISUALIZATION

# ================================================================

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.metrics import confusion_matrix cm = confusion_matrix(y_test, y_pred)

# ================================================================

# ROC CURVE PLOT

# ================================================================

from sklearn.metrics import roc_curve, auc def federated_predict_proba(models, X):

    probs = np.zeros((len(X), 2))     for m in models:

        probs += m.predict_proba(X)     return probs / len(models)

y_probs = federated_predict_proba(local_models, X_test)[:, 1] fpr, tpr, _ =

roc_curve(y_test, y_probs) roc_auc = auc(fpr, tpr) plt.figure(figsize=(6,5))
```

```
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {roc_auc:.6f})')

plt.plot([0, 1], [0, 1], 'k--')

 plt.xlabel('False Positive Rate')

plt.ylabel('True Positive Rate')

plt.title('ROC Curve – Federated Ensemble IDS')

plt.legend(loc='lower right')

plt.grid()

plt.show()
```

Appendix B - Screenshots



```
Accuracy: 0.9999507898233355
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       471
           1       1.00      1.00      1.00     40171

    accuracy                           1.00     40642
   macro avg       1.00      1.00      1.00     40642
weighted avg       1.00      1.00      1.00     40642
```

Fig 3: Accuracy Score

```
Confusion Matrix:
[[  471     0]
 [    2 40169]]

Performance Metrics:
True Positives (TP): 40169
True Negatives (TN): 471
False Positives (FP): 0
False Negatives (FN): 2
False Positive Rate (FPR): 0.000000
False Negative Rate (FNR): 0.000050
ROC-AUC Score: 1.000000
Matthews Correlation Coefficient (MCC): 0.997859
```

Fig 4: Classfication Report of Performace Metrics

```
Total Attacks in Test Data: 40171
Detected Attacks (True Positives): 40169
Missed Attacks (False Negatives): 2
```

Fig 5: Detected Attacks

```
Normal Traffic Correctly Identified (TN): 471
False Alarms (FP): 0
```

Fig 6: Counts of Normal Traffic Identified

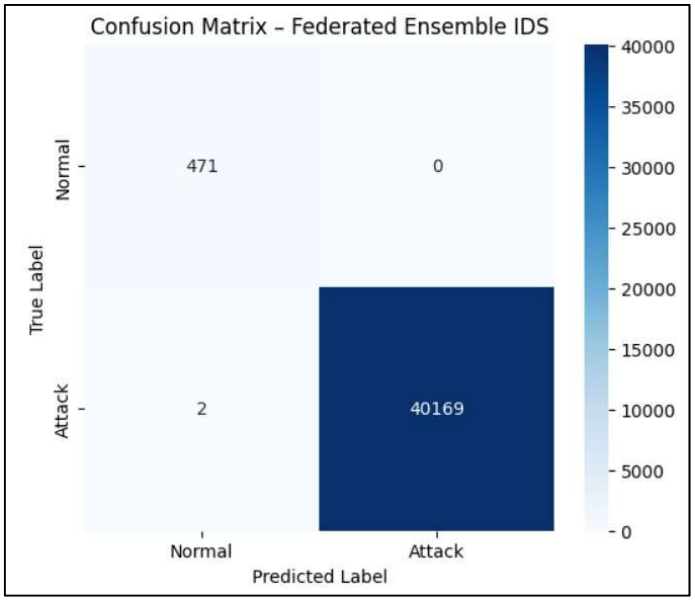


Fig 7: Confusion Matrix Visualization

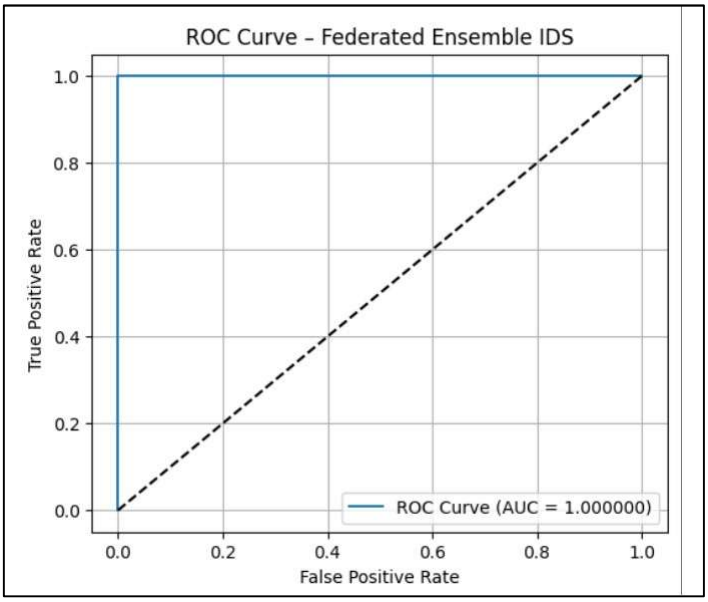


Fig 7: ROC Curve

REFERENCES

[1] Y. Afaq and S. V. Akram, “Integration of deep learning with edge computing on progression of societal innovation in smart city infrastructure: A sustainability perspective,” Sustainable Futures, vol. 9, p. 100761, 2025. doi: 10.1016/j.sftr.2025.100761.

[2] Khan, Arif Ullah & Khan, Fawad & Nadeem, Aamir & Arif, Muhammad & Bashir, Muhammad &
Choi, Wooyeol. (2025). 6G Internet-of-Things assisted smart homes and buildings: Enabling technologies, opportunities and challenges. Internet of Things. 32. 101658. 10.1016/j.iot.2025.101658.

[3] Zhou, Jingze & Alkhalaf, Salem & Abdel-Khalek, S. & Nazir, Shah. (2025). Big data analytics for smart home energy management system based on IOMT using AHP and WASPAS. Big Data Research.
41. 100534. 10.1016/j.bdr.2025.100534.

[4] Barros, E.B.C. & Souza, Wesley & Costa, Daniel & Filho, G.P. & Figueiredo, Gustavo & Peixoto, Maycon. (2024). Energy management in smart grids: An Edge-Cloud Continuum approach with Deep Q-learning. Future Generation Computer Systems. 165. 107599. 10.1016/j.future.2024.107599.

[5] A. Boumaiza, “Q-GRID SMART: A blockchain-enabled smart home energy management and analytics system,” Results in Engineering, vol. 28, p. 107093, 2025. doi: 10.1016/j.rineng.2025.107093.

[6] Choudhry, Aryan & Gouda, Sunil & Satpathy, Surya & Shukla, Krishna & Dash, Arya & Pasayat, Ajit. (2025). Integration of EEG-based BCI Technology in IoT enabled Smart Home Environment: An in-depth comparative analysis on Human-Computer Interaction Techniques. Expert Systems with Applications. 294. 128730. 10.1016/j.eswa.2025.128730.

[7] Kumar, G. & Reddy, Sivananda & Saxena, Sugandha & Swamy, K. & P., Ashwini & Kumar, U. Pavan. (2025). FL-DPCSA: Federated learning with differential privacy for cache side-channel attack detection in edge-based smart grids. e-Prime - Advances in Electrical Engineering, Electronics and Energy. 13. 101057. 10.1016/j.prime.2025.101057.

[8] Cai, Taoyang & Ma, Jingfei & Ge-Zhang, Shangjie. (2025). Smart Cities and Smart Health:
Innovations in Home Medical Devices for Efficient Healthcare Delivery. Results in Engineering. 27. 105739. 10.1016/j.rineng.2025.105739.

[9] Alajramy, Loay & simoni, Marco & Rasori, Marco & Saracino, Andrea & Mori, Paolo. (2025). OnDevice Derivation of IoT Usage Control Policies: Automating U-XACML Policy Generation from Natural Language with LLMs in Smart Homes Environments. Future Generation Computer Systems. 175. 10.1016/j.future.2025.108067.

[10] Li, Wenda & Yigitcanlar, Tan & Nili, Alireza & Browne, Will & Li, Fei. (2025). Responsible Smart Home Technology Adoption: Exploring Public Perceptions and Key Adoption Factors. Internet of Things. 32. 101622. 10.1016/j.iot.2025.101622.
[11] A. A. Rawahi and M. A. Wahaibi, "Analyze user behavior to improve smart home services," Procedia Computer Science, vol. 258, pp. 2006–2017, 2025. doi: 10.1016/j.procs.2025.04.451.
[12] Li, Wenda & Yigitcanlar, Tan & Nili, Alireza & Browne, Will & Li, Fei. (2025). Responsible Smart Home Technology Adoption: Exploring Public Perceptions and Key Adoption Factors. Internet of Things. 32. 101622. 10.1016/j.iot.2025.101622.

[13] Oulefki, Adel & Kheddar, Hamza & Amira, Abbes & Kurugollu, Fatih & Himeur, Yassine. (2024).
Innovative Ai Strategies for Enhancing Smart Building Operations Through Digital Twins: A Survey.
10.2139/ssrn.5015571.
[14] Lin, Yu-Hsiu & Ciou, Jian-Cheng. (2024). A Privacy-Preserving Distributed Energy Management Framework Based on Vertical Federated Learning-Based Smart Data Cleaning for Smart Home Electricity Data. Internet of Things. 26. 101222. 10.1016/j.iot.2024.101222.
[15] Lin, Yu-Hsiu & Ciou, Jian-Cheng. (2024). A Privacy-Preserving Distributed Energy Management Framework Based on Vertical Federated Learning-Based Smart Data Cleaning for Smart Home Electricity Data. Internet of Things. 26. 101222. 10.1016/j.iot.2024.101222.
[16] Rey-Jouanchicot, Jordan & Campo, Eric & Bouraoui, Jean-Léon & Bottaro, Andre & Vigouroux, Nadine & Vella, Frédéric. (2025). Adaptation in Smart Home Automation Systems: A systematic review of decision-making and interaction. Internet of Things. 31. 101588. 10.1016/j.iot.2025.101588.
[17] Senapati, Tapan & Chen, Guiyun & Pedrycz, Witold. (2025). Artificial intelligence-driven energy optimization in smart homes using interval-valued Fermatean fuzzy Aczel-Alsina aggregation operators. Journal of Building Engineering. 105. 112418. 10.1016/j.jobe.2025.112418.
[18] Nguyen, Huy-Trung & Pham, Van-Ha & Vu, Viet-Thang & Nguyen, Trung. (2025). Web access as a service for CSA matter protocol: Bridging matter and the W3C Web of Things framework for smart home interoperability. Internet of Things. 32. 101618. 10.1016/j.iot.2025.101618.
[19] Hassan, Ali H & Ahmed, Elsadig & Hussien, Jamal & Sulaiman, Riza & Abdulhak, Mansoor & Kahtan, Hasan. (2025). A cyber physical sustainable smart city framework toward society 5.0:
Explainable AI for enhanced SDGs monitoring. Research in Globalization. 10. 100275. 10.1016/j.resglo.2025.100275.
[20] Yao, Aiting & Pal, Shantanu & Li, Gang & Li, Xuejun & Zhang, Zheng & Jiang, Frank & Dong, Chengzu & Xu, Jia & Liu, Xiao. (2025). FedShufde: A privacy preserving framework of federated learning for edge-based smart UAV delivery system. Future Generation Computer Systems. 166. 107706. 10.1016/j.future.2025.107706.

[21] Zahid, Herman & Zulfiqar, Adil & Adnan, Muhammad & Iqbal, Muhammad & Shah, Anwar & Abbasi, Muhammad & Gasim M. Hassan, Salah Eldeen. (2025). Transforming Nano Grids to Smart Grid 3.0: AI, Digital Twins, Blockchain, and the Metaverse Revolutionizing the Energy Ecosystem. Results in Engineering. 27. 105850. 10.1016/j.rineng.2025.105850.
[22] G. Castanho, H. Taherdoost, and M. Madanchian, "AI-powered sustainability in smart cities," Procedia Computer Science, vol. 258, pp. 2639–2646, 2025. doi: 10.1016/j.procs.2025.04.525.
[23] Lu, Yao & Vijayananth, Vishalini & Perumal, Thinagaran. (2025). Smart Home Energy Prediction Framework using Temporal Kolmogorov-Arnold Transformer. Energy and Buildings. 335. 115529.
10.1016/j.enbuild.2025.115529.
[24] Wang, Han & Anurag, Kumar & Amira Rayane, Benamer & Arora, Priyansh & Wander, Gurleen
& Johnson, Mark & Anjana, Ranjit & Mohan, Viswanathan & Gill, Sukhpal Singh & Uhlig, Steve & Buyya, Rajkumar. (2025). HealthAIoT: AIoT-driven smart healthcare system for sustainable cloud computing environments. Internet of Things. 31. 101555. 10.1016/j.iot.2025.101555.
[25] Zhao, Shijiao & Chen, SiZhuo & R. Alsenani, Theyab & Alotaibi, Badr & Abuhussain,
Mohammed. (2025). Residential energy consumption and price forecasting in smart homes based on the internet of energy. Sustainable Energy Technologies and Assessments. 73. 104081.
10.1016/j.seta.2024.104081.

Project Report Evaluation form

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (2)	Poor (1)
Report Formatting	Fully formatted as per guidelines	Minor formatting issues	Moderate formatting issues	Several formatting issues	Not formatted
Grammar and Language	No grammatical/spelling errors	Few minor errors	Noticeable errors	Frequent errors	Poor grammar, hard to understand
Referencing, Tables & Figures	All cited properly, visuals are clear	Minor citation/visual issues	Some missing citations or unclear visuals	Many missing or incorrect citations	No citations; visuals poorly presented
Technical Content Quality	In-depth, innovative, and well-explained with Original content	Clear and relevant	Meets basic expectations	Lacks depth or clarity	Incomplete or incorrect technical info

Name	**Joshua Suriyan N**
Reg. No	**URK22CS1035**
Report Correctly formatted	
No Grammatical Errors	
All References are Cited Inside; Tables and Figures Represented Properly	
Technical Content	
Total	

Guide Signature